

3. Control Bus Servo

3. Control Bus Servo

- 3.1 Experiment Objective
- 3.2 Experiment Preparation
- 3.3 Experiment Procedure
- 3.4 Experiment Summary

3.1 Experiment Objective

This course focuses on learning how to control the rotation of the robot's eighteen degrees-of-freedom servos.

3.2 Experiment Preparation

This course involves the following functions from the Muto hexapod robot's Python library:

motor(servo_id, angle, runtime=100): Controls the rotation of the robot's eighteen servos.

The `servo_id` parameter represents the ID number of the servo to be controlled. Each servo has a unique ID. The value range for `servo_id` is [1, 18], corresponding to the 18 bus servos on the robot.

The `angle` parameter represents the target angle for the servo's movement, with a value range of [-90, 90].

The `runtime` parameter is optional, with a default value of 100. It represents the time for the servo's movement. Within the valid range, a smaller value results in a faster rotation speed.

Servo_torque_on(servo_id=0): Enables the servo torque.

Servo_torque_off(servo_id=0): Disables the servo torque.

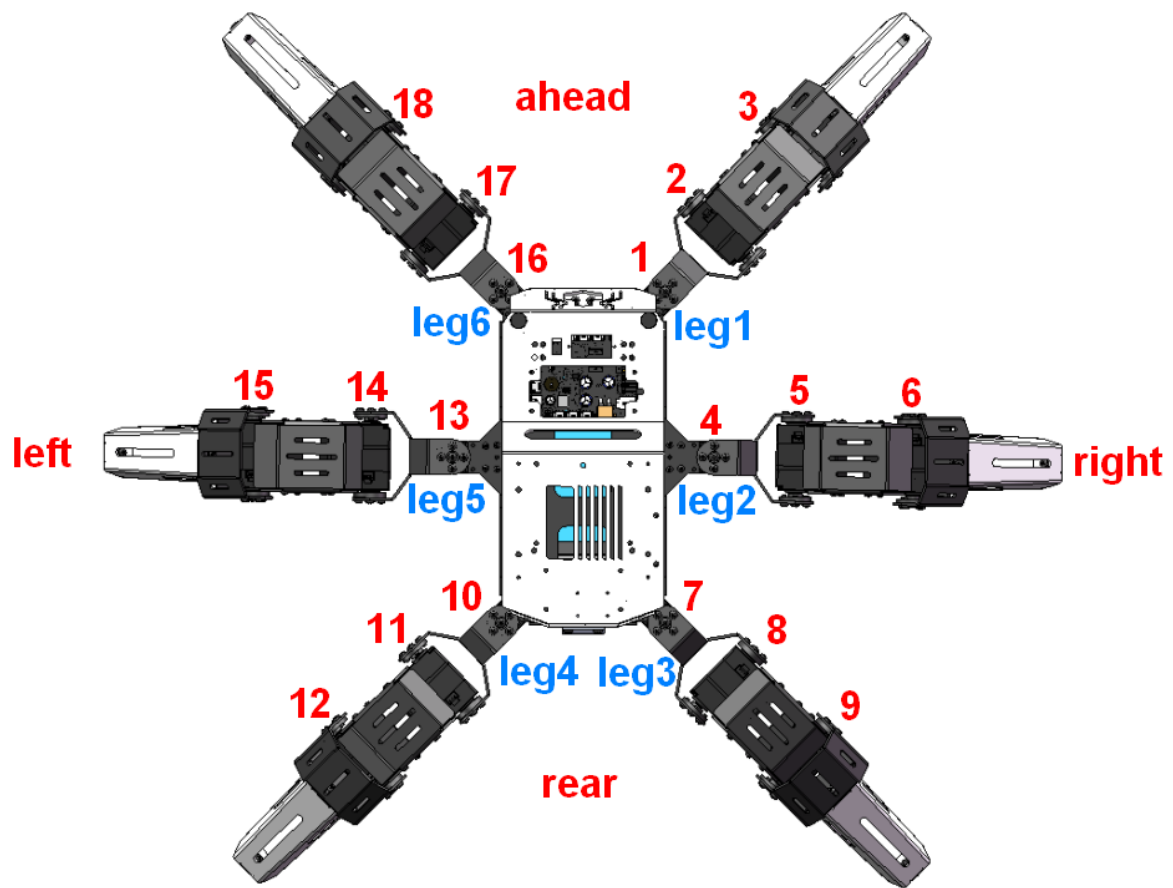
The optional `servo_id` parameter represents the ID of the servo to be controlled, with a value range of [0, 18]. When `servo_id=0`, it controls the torque for all servos (on or off). When `servo_id` is in the range [1, 18], it controls the torque for the corresponding servo ID.

load_leg(leg): Enables the torque for the three servos on a specific leg.

unload_leg(leg): Disables the torque for the three servos on a specific leg.

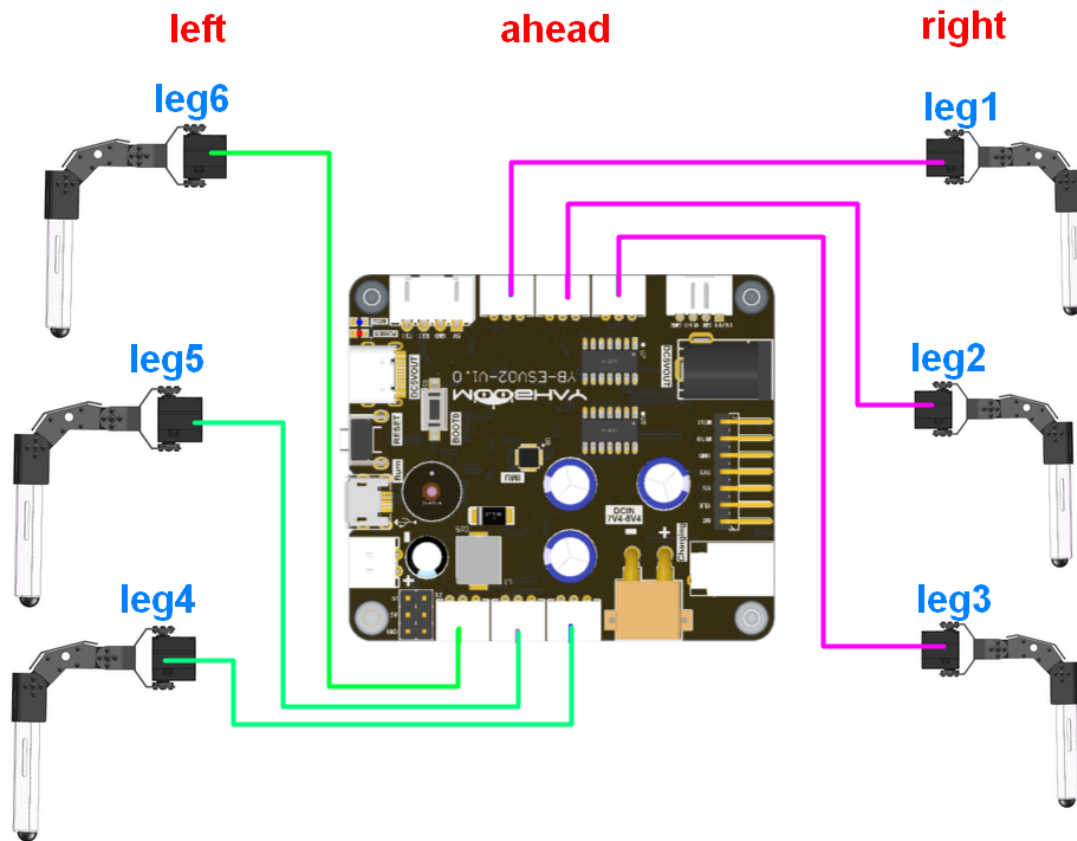
The `leg` parameter represents the ID of the leg to be controlled, with a value range of [1, 6], corresponding to the robot's six legs.

The Muto hexapod robot has a total of six legs, with three bus servos on each leg, making a total of eighteen bus servos, assigned IDs from 1 to 18. The leg servo ID assignment is shown in the figure below:



The six legs of the Muto hexapod robot are divided into two groups: leg1, leg2, and leg3 form the right-side group, while leg4, leg5, and leg6 form the left-side group. Within the same group, the wiring order for the three legs does not matter, but the left-side legs must be plugged into the corresponding left-group interfaces and not into the right-leg interfaces. The left-leg group interfaces are the three interfaces near the PWM servo interface, and the right-leg group interfaces are the three interfaces near the DC5V interface.

Leg connection diagram:



3.3 Experiment Procedure

Open the JupyterLab client and navigate to the code path:

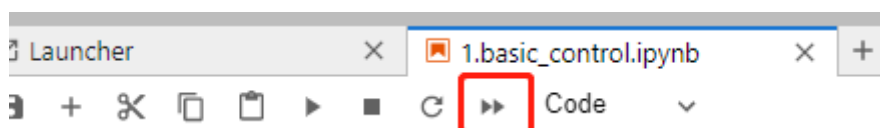
```
muto/Samples/Control/3.control_motor.ipynb
```

By default, `g_ENABLE_CHINESE` is `False`. If you need to display Chinese, please set `g_ENABLE_CHINESE=True`.

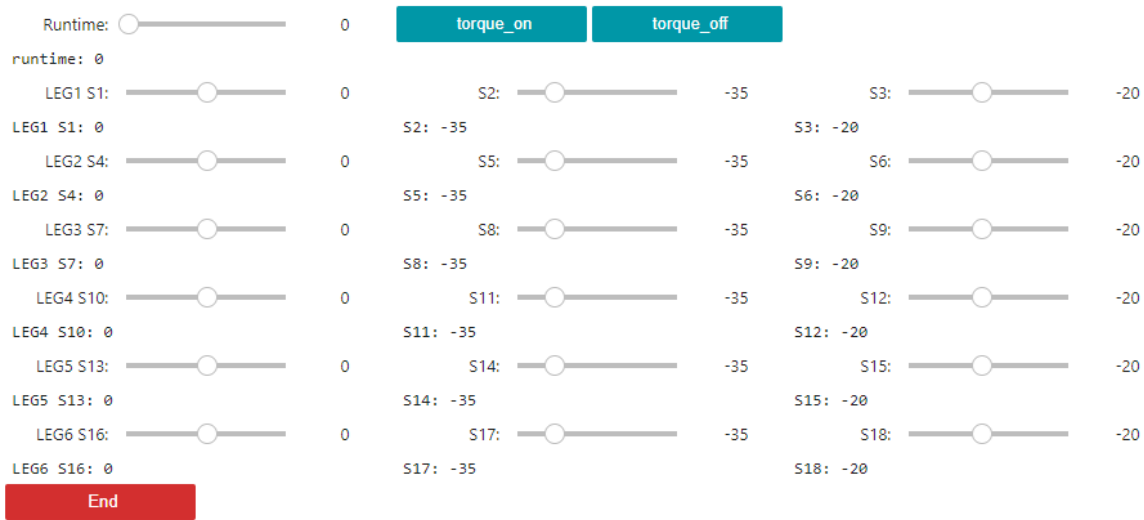
```
# Chinese switch. The default value is English
g_ENABLE_CHINESE = False

Name_widgets = {
    'End': ("End", "Finish"),
    'torque_on': ("torque_on", "Torque On"),
    'torque_off': ("torque_off", "Torque Off")
}
```

Click 'Run All Cells', then scroll to the bottom to see the generated controls.



Operate the different controls to manage their respective functions.



The Runtime slider mainly controls the movement time of the bus servos.

```
def on_slider_runtime(value):
    global g_bot, g_runtime
    g_runtime = value
    print("runtime:", value)
```

Based on the servo's ID number, the servos of each leg are controlled by a slider. Changing the value of the corresponding slider controls the servo with the corresponding ID.

```
def on_slider_S1(angle):
    print("LEG1 S1:", angle)
    g_bot.motor(1, angle, g_runtime)
def on_slider_S2(angle):
    print("S2:", angle)
    g_bot.motor(2, angle, g_runtime)
def on_slider_S3(angle):
    print("S3:", angle)
    g_bot.motor(3, angle, g_runtime)
```

The event handling for the buttons to enable and disable servo torque is as follows.

```
# Key press event processing
def on_button_clicked(b):
    with output:
        print("Button clicked:", b.description)
        if b.description == Name_widgets['torque_on'][g_ENABLE_CHINESE]:
            g_bot.Servo_torque_on()
        elif b.description == Name_widgets['torque_off'][g_ENABLE_CHINESE]:
            g_bot.Servo_torque_off()
```

3.4 Experiment Summary

In this experiment, JupyterLab controls are used to manage the hexapod robot's bus servos. The servos are already limited in the program, and you can control their movement by sliding the corresponding ID's slider.

When you need to disable the servo torque, click 'torque_off' to turn off the torque. At this point, you cannot control the servos with the sliders, but you can move them by hand. Click the 'torque_on' button to restore the servo torque.



To exit the program, press the 'End' button.